

UCSH-201

Sin

GRAPH ADT

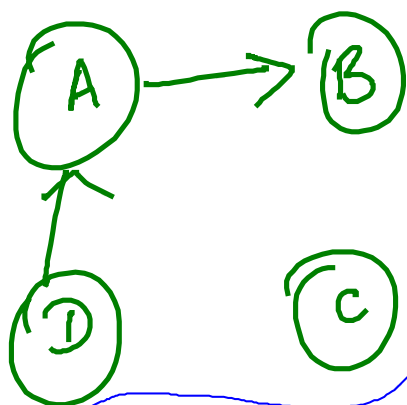
Graph : Collection of nodes, called Vertices,
and line segments, called Arcs or edges,

that connect pairs of nodes

Graph
↙ Directed

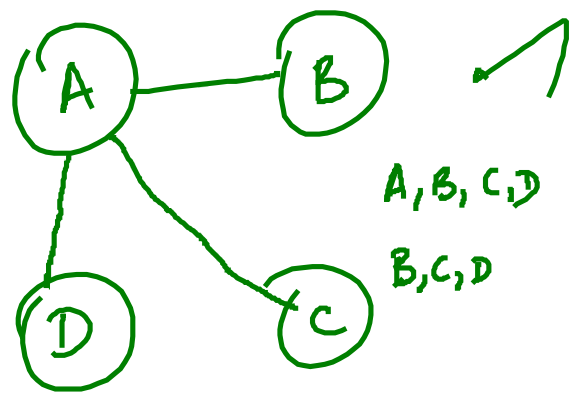
Each arc (line) has
a direction

Def



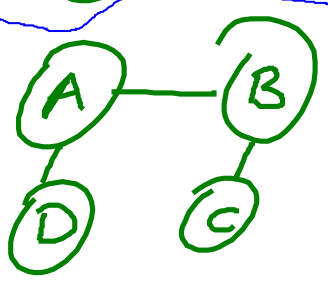
A, B, C

A, B, C, D



A, B, C, D

B, C, D



A is adjacent to B.

B is adjacent to A.

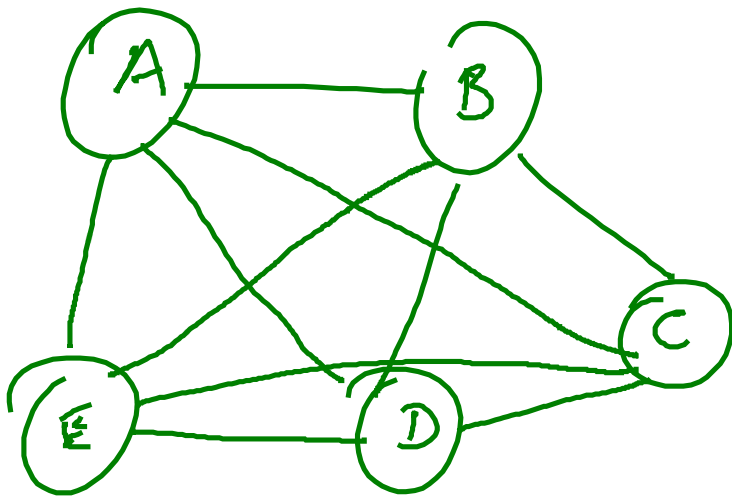
Cycle: Path consisting of at least 3 vertices that starts and ends with the same vertex.

Connected: Two vertices are said to be connected if there is a path between them.

Strongly Connected: A Directed graph is strongly connected if there is a path from each vertex to every other vertex.

Weakly Connected: A Directed graph is weakly connected if at least two vertices are not connected.

Disjoint Graph: A graph that is not connected.

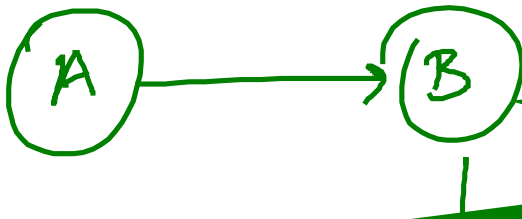


Complete

Edge

Vertex

Other



A

C

Degree of a Vertex: No. of arcs incident

Outdegree of a Vertex (digraph) = No. of arcs leaving a Vertex.

Indegree of a Vertex (digraph) = No. of arcs entering a Vertex.

Every Tree is a graph

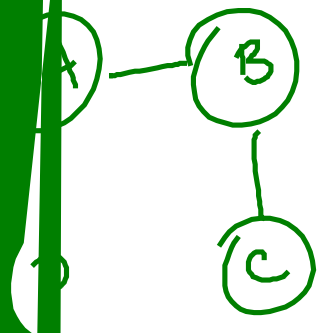
But Every graph need not be a tree.

Primitive Graph Operation

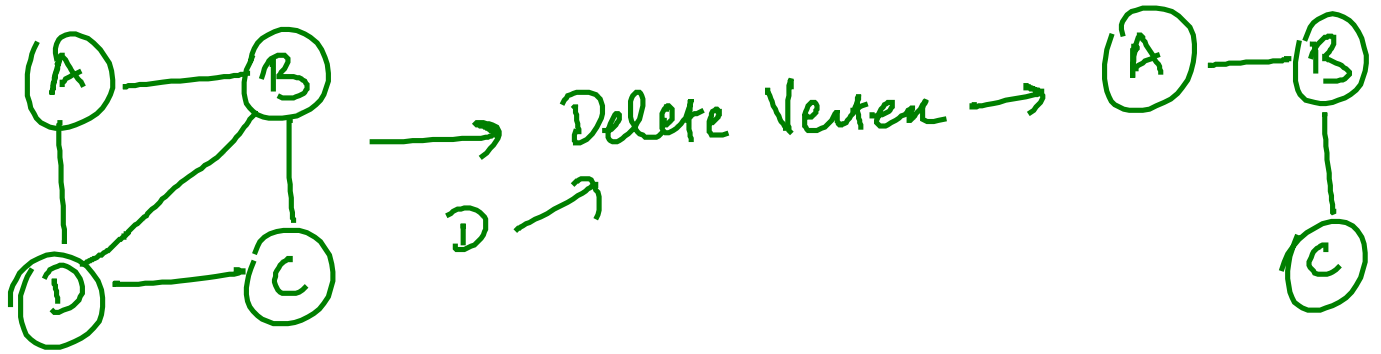
1. Insert Vertex



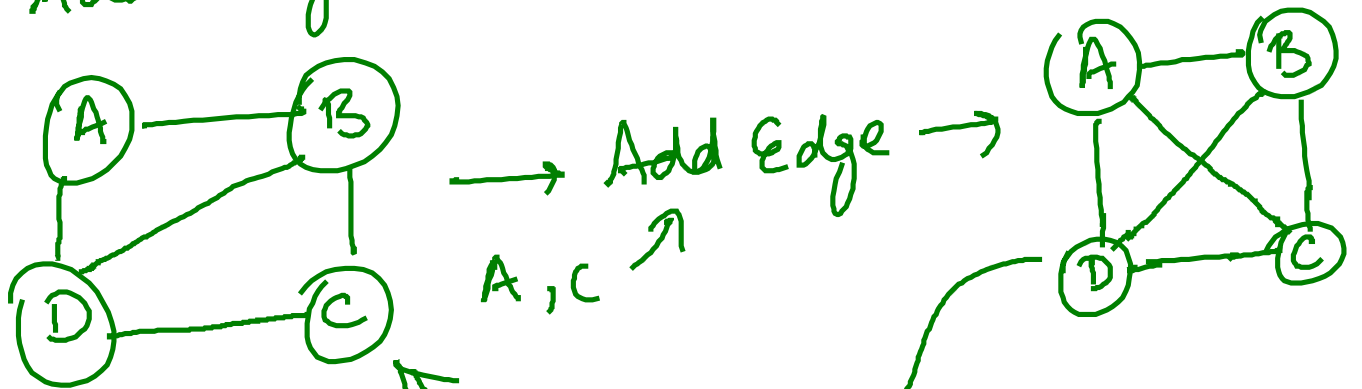
Insert



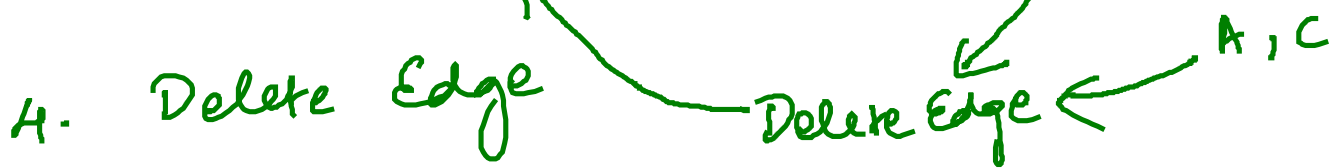
2. Delete Verten



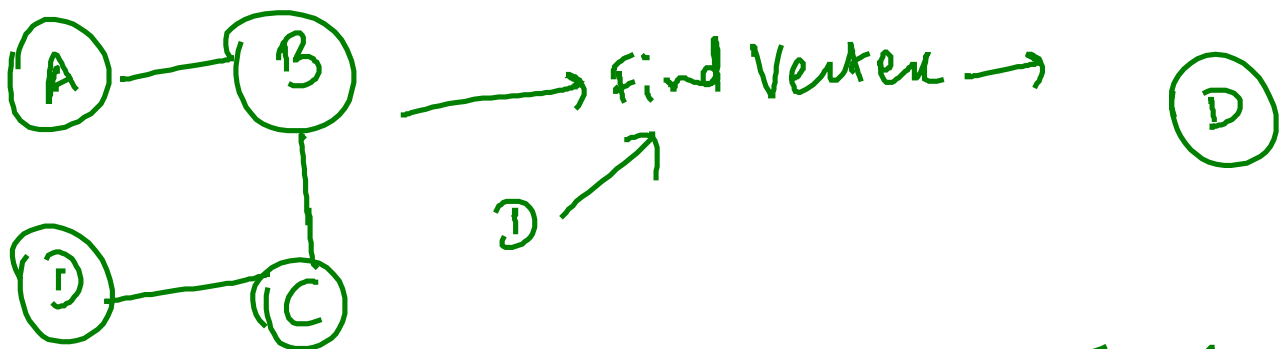
3. Add Edge



4. Delete Edge



5. Find Verten



6. Traverse Graph

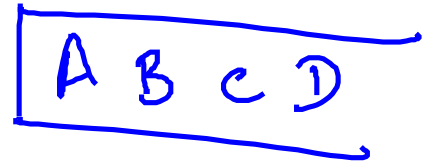
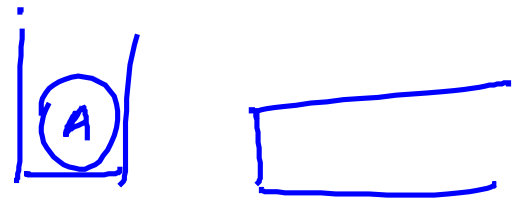
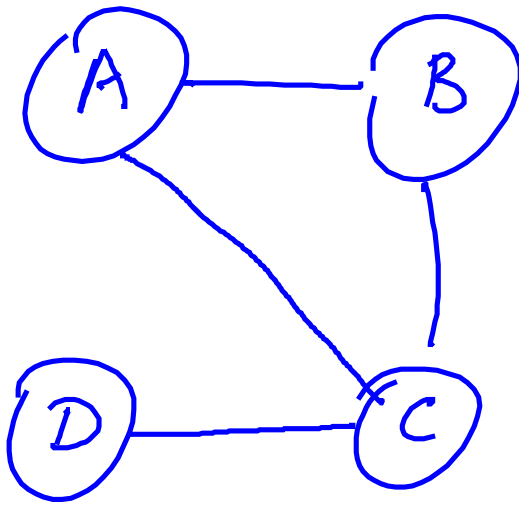
→ Depth First
→ Breadth First

1. Traversal :

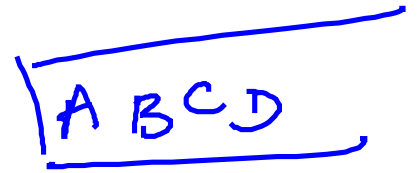
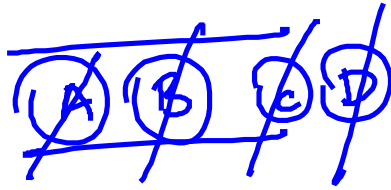
in (stack,)

Wrt insert (stack)

{



Q → BF



First Traversal :

(, A)

While (! isEmpty())

{ v = ()

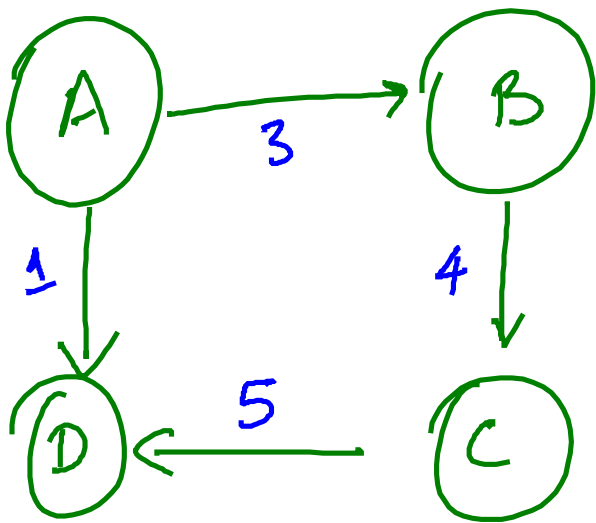
Process (v)

for each w adj

Graph Storage Structures

→ Adjacency Matrix

Uses an array for Vertices and a 2-dimensional array (matrix) for edges (arcs).

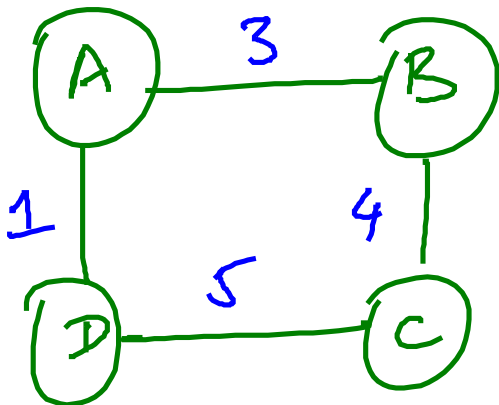


Vertices :

0	1	2	3
A	B	C	D

Edges :

	0	1	2	3
0	?	3	0	1
1	0	?	1	0
2	0	0	?	1
3	0	0	0	?



Limitations : Storage - ? $O(N^2)$
Size of graph to be known ahead.

can have only

on

then list

mei-

se

at

).

Can

1

2

3

4



struct Vertex

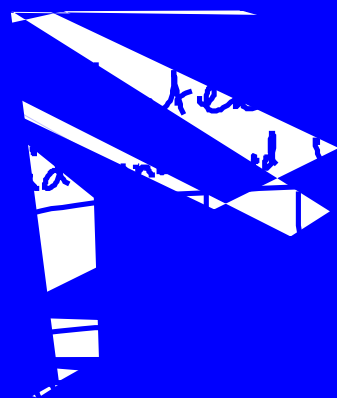
```
{ int id;  
  int inDegree;  
  int outDegree;  
  bool processed;  
  struct Vertex *next;  
  struct arc *first;  
};
```

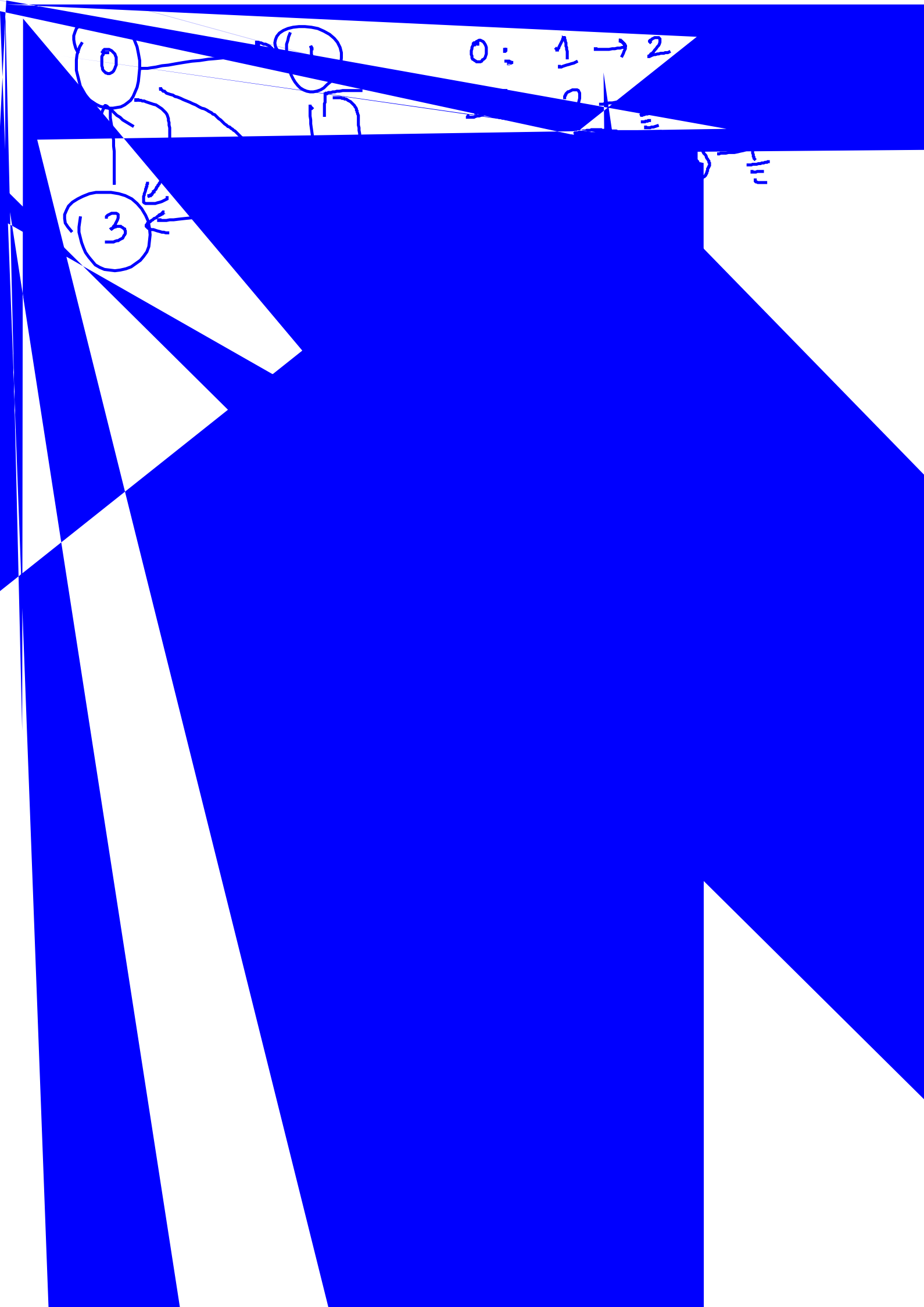
struct arc

```
{ int weight;  
  struct Vertex *destination;  
  struct arc *next;  
};
```

struct Graph * createGraph()

```
{ struct Graph *g = NULL;  
  g = (struct Graph *) malloc(sizeof(struct Graph));  
  g->count = 0;      g->first = NULL;  
  return(g);  
}
```





0

1

$0: \underline{1} \rightarrow 2$

0

$\frac{1}{17}$

$\frac{1}{17}$

3

struct Vertex * CreateVertex (int id)

```
{  
2 struct Vertex *v = NULL;  
  v = (struct Vertex *) malloc (sizeof (struct Vertex));  
  
  v → id = id;  
  v → indegree = v → outdegree = 0;  
  v → processed = false;  
  v → next = NULL;  
  v → first = NULL;  
  return (v);  
}
```

struct Arc * CreateArc (int w)

```
{  
2 struct Arc *a = NULL;  
  a = (struct Arc *) malloc (sizeof (struct Arc));
```

a → weight = w ;

a → d

```

struct Vertex * find (struct Graph *g, int id)
{
    struct Vertex *t = NULL;
    if (g == NULL)
        return (NULL);
    t = g → first;
    while (t != NULL)
    {
        if (t → id == id)
            return (t);
        t = t → next;
    }
    return (NULL);
}

```

```

void addVertex (struct Graph *g, int id)
{
    struct Vertex *v = NULL, *t = NULL;
    if (g == NULL)
        return;
    v = find (g, id);
    if (v != NULL)
    {
        printf ("Vertex id already exists\n", id);
        return;
    }
}

```

```

v = createVertex(id);
g → Count = g → Count + 1;
if (g → first == NULL)
{
    g → first = v;
    return;
}

```

```

if (id < g → first → id)
{
    v → next = g → first;
    g → first = v;
    return;
}

```

```

t = g → first;

```

```

while (t → next != NULL &&
        t → next → id < id)

```

```

    t = t → next;

```

```

v → next = t → next;

```

```

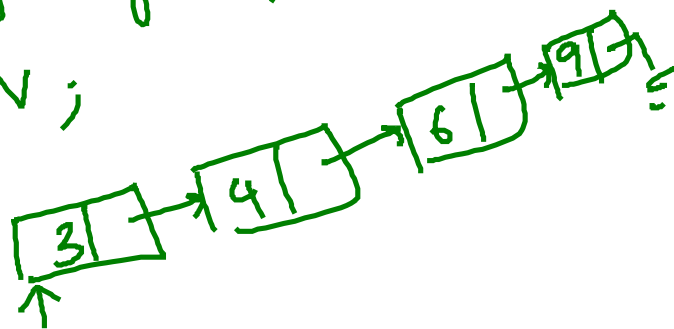
t → next = v;

```

```

}

```



```
void addArc (struct Graph *g, int from,  
            int to, int w)
```

```
{  
    struct Vertex *src = NULL, *dst = NULL
```

```
    src = find (g, from);
```

```
    if (src == NULL)
```

```
{  
    printf ("Source vertex not found\n");  
    return;
```

```
    dst = f
```

```
if (src → first == NULL)
```

```
{ src → first = a ;
```

```
return ;
```

```
}
```

```
if ( w ≤ src → first → weight )
```

```
{
```

```
    a → next = src → first ;
```

```
    src → first = a ;
```

```
    return ;
```

```
}
```

```
t = src → first ;
```

```
while ( t → next != NULL &&
```

```
        t → next → weight ≤ w )
```

```
    t = t → next ;
```

```
a → next = t → next ;
```

```
t → next = a ;
```

```
}
```